

CREST: Deployment-Realistic Hardware-in-the-Loop NAS for Embedded Sensing Systems

Anonymous Author(s)

Abstract

Deploying neural networks on low-power microcontrollers (MCUs) requires selecting model architectures under tight memory, latency, and energy constraints. Existing workflows often simplify this process along one or more axes: static proxy costs such as FLOPs or parameters, treating one MCU as representative, and continuous-inference tests instead of deployed sensing schedules. These assumptions can mis-rank Pareto-front candidates, miss infeasible deployments, and obscure schedule-dependent energy.

We present CREST (Cross-platform Runtime Evaluation and Search Tool), a deployment-realistic hardware-in-the-loop (HIL) neural architecture search (NAS) framework for MCU sensing systems. CREST keeps the optimizer, HIL measurement boundary, logging, and replay workflow fixed while exposing workload, model family, target backend, schedule, quantization, and scoring policy as configurable axes. This makes deployment effects experimentally separable within one reusable workflow.

We evaluate CREST on inertial odometry and audio classification across three Arm Cortex-M targets. For inertial odometry, measured-energy HIL search reduces median per-inference energy by 41.7% versus FLOPs-based selection and 40.8% versus memory-traffic-based selection at similar error. FLOPs-based selection also chooses infeasible deployments on memory-constrained targets. On the STM32 N657 target, continuous-inference and duty-cycled searches produce different Pareto frontiers. For audio classification, the same application-level policy selects different DS-CNN architectures on different boards, and cross-board replay changes deployment cost substantially.

Overall, CREST shows that deployment-realistic MCU NAS must jointly optimize model architecture, target platform, runtime schedule, and deployment policy rather than relying only on static proxy costs or continuous-inference measurements.

CCS Concepts

• **Computer systems organization** → **Embedded software**;
• **Computing methodologies** → **Neural networks**; *Machine learning*; • **Software and its engineering** → *Software libraries*.

Keywords

Neural architecture search, TinyML, hardware-in-the-loop, embedded machine learning, energy measurement

ACM Reference Format:

Anonymous Author(s). 2027. CREST: Deployment-Realistic Hardware-in-the-Loop NAS for Embedded Sensing Systems. In *Proceedings of ACM/IEEE International Conference on Embedded Artificial Intelligence and Sensing Systems (SenSys '27)*. ACM, New York, NY, USA, 14 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Resource-constrained microcontrollers (MCUs) are increasingly the deployment target for neural network inference in sensing applications, spanning workloads such as inertial odometry [40], vibrometry [24], digital biomarkers [21], audio classification [35], and keyword spotting [11, 50]. Mature embedded inference runtimes such as TensorFlow Lite Micro [15] and CMSIS-NN [26], alongside recent Cortex-M cores with low-precision arithmetic support [7, 8], have made neural-network inference feasible within hundreds-of-KB to MB-scale MCU SRAM [5, 6, 44]. This maturity has also brought fragmentation. MCU vendors ship increasingly diverse cores and accelerators, sensing applications span a widening range of workloads, and deployment schedules range from continuous inference to deeply duty-cycled operation. Each axis of fragmentation increases the gap between how a model architecture is selected and how it performs once deployed.

This fragmentation changes the design question facing users. Selecting an MCU model is no longer only a question of finding a small architecture. It is also a deployment problem, and the right answer depends on the model, target platform, runtime schedule, and application constraints. The same model architecture can satisfy a latency, memory, or battery-lifetime constraint under one deployment configuration and violate it under another.

This paper presents CREST, a hardware-in-the-loop NAS framework for studying these deployment choices under one fixed search and measurement workflow.

Simplifying assumptions. Current hardware-aware model-selection workflows often simplify this deployment problem in three ways. First, they use proxy costs such as FLOPs, MACs, parameter counts, or tensor traffic in place of measured deployment cost. These proxies are useful for pruning, but they do not capture target-specific board and software-stack effects. Prior MCU studies show that such effects can mis-rank model architectures with similar quality [27, 31]. Second, they treat one measured platform as representative of a broader deployment class. In practice, the deployment target is the full board and software stack, not only the processor core. Memory capacity, runtime kernels, compiler choices, and power management can all change latency, energy, and feasibility [36]. Third, they measure continuous inference and assume it represents a sensing deployment. Sensing systems usually release inference on a schedule and spend the remaining window in sleep or other non-inference phases. Latency is therefore an active-time constraint, while energy is integrated over the deployment window as a whole [12].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SenSys '27, New York, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

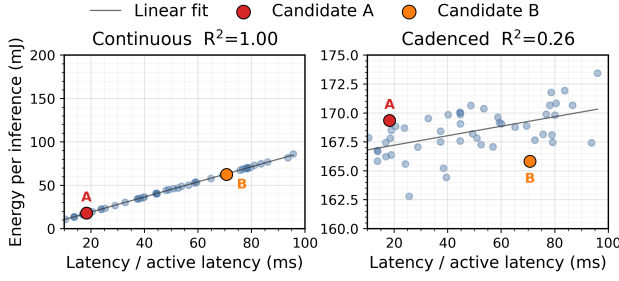


Figure 1: Model architecture latency and energy on STM32 N657 under continuous and 200 ms cadenced execution. The highlighted architectures reverse their energy ordering across runtime schedules.

We use *cadenced execution* to mean a duty-cycled sensing schedule that releases one inference every fixed period, then waits or sleeps until the next release. Figure 1 compares continuous and 200 ms cadenced execution on STM32 N657. Under continuous inference, energy per inference closely tracks active latency. Under cadenced execution, energy is measured over the full release window, which changes the ordering of candidate architectures. Architecture A uses less energy than Architecture B under continuous inference, but more energy under the cadenced schedule. The same model pair can therefore lead to different selection decisions depending on the runtime schedule.

These assumptions are the deployment axes that CREST makes configurable. By keeping the search loop, HIL measurement path, logging, and replay workflow fixed, CREST lets the evaluation attribute model-selection differences to policy choice, target platform, runtime schedule, and workload rather than to a rewritten pipeline.

Missing configuration axes. Recent hardware-aware neural architecture search (NAS) systems have moved beyond proxy-only estimates by compiling candidate models, flashing them to a device under test, and measuring deployment metrics directly during search [39, 40]. Prior HIL NAS systems such as TinyOdom demonstrated the value of direct target interaction for one vertically integrated inertial-odometry workload and deployment stack [40]. The benchmarking community has likewise moved toward multi-board characterization [9, 19, 47]. What neither trajectory has addressed is how architecture selection changes across workload, target, and schedule. Existing hardware-aware NAS systems commit, at design time, to a single target board, a single execution schedule, and a single task family hardcoded into the implementation. CREST asks a different question: how can deployment-sensitive NAS itself be made configurable across workloads, targets, schedules, and policies? To our knowledge, no existing system combines configurable task and model interfaces, multi-vendor hardware-in-the-loop measurement, execution-schedule variation, and search-driven architecture selection in one reusable MCU sensing workflow.

Challenges. Building a framework that supports this exploration requires addressing three structural challenges:

- ❶ **Vendor-specific deployment paths.** MCU toolchains use different compilation, deployment, telemetry, and power-measurement interfaces. A unified search loop must accommodate these differences without sacrificing per-vendor measurement fidelity.

- ❷ **Schedule-aware measurement.** Execution schedule is not exposed as a configurable parameter in existing NAS frameworks. Treating schedule as a first-class evaluation variable requires a measurement harness that instruments cadenced execution correctly.

- ❸ **Reusable workload and backend boundaries.** Existing systems are built as monolithic implementations rather than reusable frameworks. TinyOdom [40] is bound to inertial odometry, MCUNet [30] to image classification, and each intermixes the search algorithm, training loop, compilation path, and measurement code in one vertical pipeline. Adding a new workload, MCU vendor, model family, or optimization policy should not require constructing a new pipeline for each combination.

Our approach. CREST (Cross-platform Runtime Evaluation and Search Tool) is built for controlled comparison. A study can vary the workload, model family, target backend, runtime schedule, quantization, or scoring policy while keeping the optimizer, measurement path, logs, and replay workflow fixed. We evaluate CREST on inertial odometry and audio classification across three Arm Cortex-M targets and show that runtime-grounded HIL measurement exposes architecture-selection differences that proxy-only, single-target, and continuous-inference NAS misses.

In summary, this work makes the following contributions:

- We present CREST, an open-source, configurable hardware-in-the-loop NAS framework¹ for experimentally isolating deployment effects while keeping the optimizer, HIL measurement boundary, logging path, and replay workflow fixed.
- CREST makes deployment effects experimentally separable. The same model representation and measurement pipeline can compare proxy-based selection against measured-energy selection, continuous inference against cadenced execution, and native-board selection against cross-board replay.
- CREST exposes runtime schedule as a search-time variable. It supports continuous inference and cadenced sensing-window execution, allowing the search to optimize for the measurement regime used by the target deployment.
- Across inertial odometry and audio classification on three Cortex-M targets, CREST exposes architecture-selection differences that are invisible to proxy-only, single-target, and single-schedule NAS workflows.

2 Related Work

Proxy and predictor-based MCU NAS. Prediction is not new to embedded systems. Software and system-level energy models have long estimated power or energy from program behavior, processor settings, platform measurements, or runtime counters [43, 48]. These models can work well in their intended setting, but they are usually tied to a processor, board, counter set, workload, or calibration method. MCU NAS therefore often uses cheaper signals such as MACs, operation counts, RAM, flash, or latency lookup tables [25, 29, 30, 37, 38]. MicroNets makes this assumption explicit by using operation count as a proxy for latency and energy within a given task/search space and target hardware/software setting [10].

¹The anonymized CREST repository is available at <https://github.com/anonymous-crest/crest>.

Table 1: Capability comparison for CREST and closely related works. ● indicates a central capability, ◐ indicates partial or borderline support, and ○ indicates absence or not central. *Measured targets* requires measurements on more than one real hardware platform. *HIL metrics* requires hardware measurements to feed back into the search or selection loop rather than serve as post-hoc analysis. *Extensible tasks/targets* requires reusable interfaces for adding workloads, models, or hardware targets rather than source-code modifications.

Work	Artifact type	Model	Target scope	Availability	Automated search	Measured targets	Proxy metrics	HIL metrics	Extensible tasks/targets	Schedule options
CREST	Full tool	Neural	Multi-vendor MCUs	Open source	●	●	●	●	●	●
TinyNS 2024 [39]	Full tool	Neurosymbolic	STM32 family	Open source	●	●	●	●	○	○
TinyOdom 2022 [40]	Full tool	Neural	STM32 family	Open source	●	●	●	●	○	○
EvoNAS 2023 [18]	Full tool	Neural	Single MCU	No public artifact found	●	○	○	●	○	○
Edge Impulse 2023 [22]	Product	Neural	MCU + CPU	Product	◐	●	●	○	●	○
MicroNAS 2025 [25]	Search stage	Neural	STM32 family	Available on request	●	◐	●	○	○	○
MicroNets 2021 [10]	Search stage	Neural	STM32 family	Open source	●	●	●	○	○	○
μNAS 2020 [29]	Search stage	Neural	Single MCU	Open source	●	○	●	○	○	○
TinyTNAS 2024 [38]	Search stage	Neural	Multi-vendor MCUs	Open source	●	●	●	○	○	○
NanoNAS 2024 [17]	Search stage	Neural	STM32 family	Open source	●	○	●	○	○	○
ReactQuant 2025 [33]	Search stage	Neural	Single MCU	No public artifact found	●	○	●	○	○	○
EdgeMark 2025 [19]	Benchmark	N/A	Multi-vendor MCUs	Open source	○	◐	○	○	○	○
MLonMCU 2023 [47]	Benchmark	N/A	Multi-vendor MCUs	Open source	○	●	○	○	●	○
HW-NAS-Bench 2021 [28]	Benchmark	N/A	CPU + accelerator	Open source	○	◐	○	○	○	○
nn-Meter 2021 [49]	Estimator	N/A	Mobile edge	Open source	○	●	○	○	○	○
InstMeter 2026 [31]	Estimator	N/A	Multi-vendor MCUs	No public artifact found	○	●	○	○	○	○

These proxies are useful for pruning search, but prior MCU studies show that kernels, operator mix, memory behavior, and framework choices can change latency and energy enough to make proxy-only search decisions unreliable [20, 26, 27].

Predictors and HIL measurement. Learned predictors can reduce search cost by estimating latency or energy from calibration data, counters, or benchmark measurements. CREST instead measures whether a candidate builds, fits in memory, meets timing constraints, and consumes energy under the selected runtime schedule. Most predictors require per-platform calibration data, hardware counters, or benchmark measurements. They typically predict inference-level cost rather than the board-level scheduled window that determines sensing energy. This gap matters for a NAS. A predictor that estimates latency or active-inference energy cannot report memory-allocation failures, backend build failures, deadline misses, sleep-path behavior, or the energy of non-inference phases. nn-Meter targets latency prediction for mobile and edge-class CPU, GPU, and VPU backends rather than MCU deployment energy [49]. InstMeter is a recent MCU predictor that reports substantially lower energy and latency error than MAC-based and nn-Meter baselines, but its main energy evaluation still reports about 30% 90th-percentile relative error with target-specific training samples. Unlike CREST, InstMeter predicts inference cost rather than a full scheduled board-level deployment window [31]. For these reasons, predictor-based NAS is not a replacement for CREST’s schedule-aware HIL architecture selection.

HIL measurement and benchmarking infrastructure. HIL NAS moves the reference measurement onto target hardware. Several HIL NAS systems move closer to deployment measurement. TinyOdom and TinyNS query target devices during search to check deployability, memory use, and latency [39, 40]. EvoNAS is a HIL evolutionary NAS system that also measures per-inference energy, but its evaluation focuses on a fixed Speech Commands workload and nRF52840 Development Kit stack [18]. These systems show the value of target interaction, but they usually fix the workload,

target stack, runtime schedule, or measurement scope. CREST instead provides a modular HIL NAS framework that varies workload, backend, schedule, quantization, and scoring policy over a fixed measurement, logging, replay, and metric path. This design makes deployment effects experimentally separable.

Benchmarking systems make embedded ML results comparable across tasks, tools, and hardware, but they serve a different role than NAS systems that use measurements to select architectures. MLPerf Tiny [9] standardizes benchmark tasks, metrics, and reporting requirements for embedded inference. MLonMCU [47] and EdgeMark [19] extend this toward multi-board characterization with retargetable build flows. HW-NAS-Bench [28] provides measured and estimated hardware costs as a substrate for NAS algorithm comparison. These systems characterize model costs across hardware variants, but they do not directly couple deployment-specific HIL measurement into architecture selection.

Schedule-aware evaluation in sensing. Embedded sensing applications have long recognized that periodic operation, not continuous throughput, characterizes deployment. Keyword spotting systems account for frame periods when reporting energy and latency [11, 50], and acoustic-scene classifiers report measurements relative to duty cycle on low-power MCU devices [34]. MicroNets [10] introduces a workload-uptime metric capturing the fraction of input stride spent in active inference, and Bartoli et al. [12] propose a phase-aware energy and latency measurement method that uses oscilloscope-observed trigger signals to separate inference from surrounding work. These contributions appear as deployment analyses, benchmark methodologies, or fixed workload assumptions rather than as a configurable runtime regime inside a search loop. In contrast, CREST exposes runtime schedule as a search-time configuration, so schedule-aware energy can change which architectures the NAS process selects rather than only how a finished model is reported.

Embedded ML runtimes and MLOps platforms. Edge Impulse [22] is a production MLOps platform for TinyML, supporting

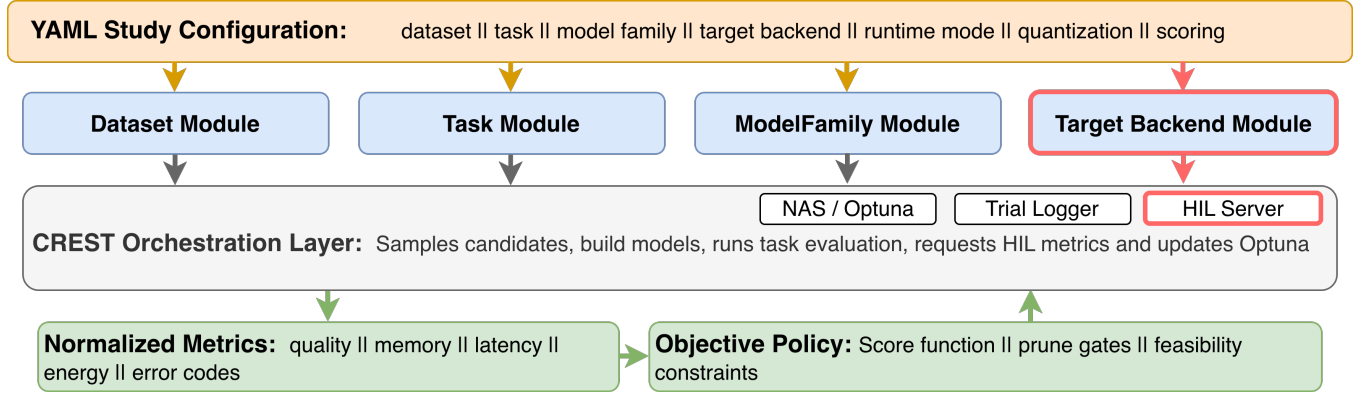


Figure 2: CREST framework structure. A study configuration selects the workload, model family, target backend, runtime mode, quantization mode, and scoring policy. The orchestration layer composes the selected interfaces, requests HIL measurements when needed, logs normalized metrics, and returns scores and feasibility feedback to the optimizer.

model tuning, multi-board deployment, and dataset management. TensorFlow Lite Micro [15] provides the embedded inference runtime that CREST and most systems above build on, including MCU-oriented memory management through a fixed arena and memory planner. These systems address key pieces of embedded ML deployment. CREST complements them as a research artifact for controlled HIL studies, where workload, target backend, measurement policy, and runtime schedule are explicit experimental variables.

Table 1 summarizes the capability landscape across these communities. CREST extends the HIL NAS work by treating workload, target, and schedule as configurable axes rather than design-time commitments. The framework couples multi-board measurement into the architecture selection loop, exposes execution schedule as a first-class search axis, and supports cross-cutting comparisons through adapter interfaces rather than a vertically integrated pipeline.

3 System Architecture

CREST is a modular hardware-in-the-loop (HIL) neural architecture search (NAS) system for MCU sensing deployments. Users configure a search by selecting the sensing workload, model family, target backend, runtime schedule, and a policy used to score model choices. The search proceeds iteratively as the optimizer proposes a model and deployment choice, CREST builds and evaluates it, and the resulting measurements are scored and returned to the optimizer.

Throughout the paper, a *study* refers to one configured NAS optimization. A study begins with choosing the deployment conditions the user wants to test. The *workload* determines the data and prediction task, the *model family* defines the architectures available to the search, the *target backend* sets the board and deployment path, and the *runtime schedule* specifies how each proposed model runs on the board. The study also includes a *policy*, which states how task and hardware measurements should be scored for the optimizer.

During the study, each optimizer step is a *trial*. A trial evaluates one *candidate*, which is a concrete model and deployment choice. In the current system, a candidate may include model hyperparameters, quantization mode, and backend-supported options such

as CPU clock. A trial produces normalized *metrics*, including task quality, proxy cost, memory, latency, energy, status, and cadence telemetry when available. The policy converts these metrics into optimizer feedback. *Feasibility* records whether the candidate satisfies constraints such as memory capacity, latency, deadline, or user-defined limits. A candidate can therefore be measured successfully and still be infeasible.

The architecture is built for controlled comparisons. A study can vary the workload, board, runtime schedule, or scoring policy. The search loop, HIL boundary, replay path, and common metric fields remain fixed. The differences across studies can then be attributed to deployment conditions instead of rewritten pipelines.

CREST supports this use case through replaceable modules with explicit interfaces, shown in Figure 2. The workload-facing interfaces own data preparation, task semantics, and model construction. The target-backend interface owns vendor-specific compilation, programming, telemetry, and power measurement. Runtime schedule is also part of the study configuration, so continuous and sensing-window execution can be selected without changing the workload or model family. Together, these boundaries address  because a new workload, board, or schedule changes the relevant module or configuration field rather than requiring a new vertical pipeline for each study.

3.1 CREST Workflow

The stack view in Figure 2 shows the modules that make up a study. The workflow in Figure 3 shows how those modules are used during optimization. CREST follows a sample-evaluate-score loop in which each trial starts from a candidate proposed by the optimizer and ends with metrics that are scored and returned as optimizer feedback.

At startup, CREST resolves the selected modules and policy from the study configuration. During a trial, the model family and task turn the proposed candidate into a model that can be trained, exported, and measured. Depending on the active policy and backend, the trial may stop after host-side task evaluation, use static model proxies, collect compile-time resource reports, or cross the HIL boundary for runtime measurements.

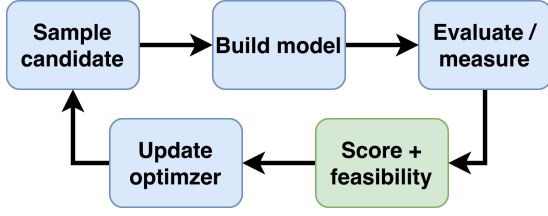


Figure 3: Conceptual NAS trial loop in CREST. Unlike weight-only optimization, each trial proposes an architecture and deployment choice, builds and evaluates the candidate, scores the resulting task and deployment metrics, and returns feedback to the optimizer.

CREST uses Optuna [2] as the optimizer and supports both scalar and multi-objective studies through the same metric interface. In a scalar study, the policy applies a user-defined score function to candidate metrics and returns one score to Optuna. In a multi-objective study, Optuna uses user-chosen metrics, such as task error and measured energy, to construct a Pareto front. In both cases, scores, objective values, feasibility information, and trial status are derived from the same set of metric fields rather than from workload-specific or backend-specific code.

3.2 System Scope

CREST ships with built-in modules for two workload/model-family pairs, three MCU backends, continuous and cadenced runtime schedules, float32 and int8 PTQ representations, proxy and HIL measurement paths, and scalar or multi-objective policies. Together, these modules let a user search and compare candidates across the deployment choices evaluated in this paper.

The same boundaries also define how CREST is extended. A new workload, model family, or target backend is added by implementing the corresponding module interface. Once the new module satisfies the interface, it uses the same optimizer loop, policy machinery, HIL boundary, replay path, and logging path as the built-in modules. This addresses the monolithic-pipeline challenge. Once the relevant modules are in place, comparing workloads, boards, schedules, or policies becomes a configuration change rather than a new source-code path.

§5 uses the configuration and extension paths in Table 2 to compare model selection across policies, boards, schedules, and tasks. The point is not that these are the only supported directions for CREST, but that each direction is exposed as a module or configuration choice rather than being fixed inside a single pipeline.

Table 2: User changes supported by CREST

User goal	What changes
Run an existing study	Select workload, backend, runtime schedule, quantization, and policy in configuration.
Add data or a task	Implement the dataset or task module that provides examples, metadata, training behavior, and quality metrics.
Add a model family	Implement the search space, candidate builder, and export model for the task output.
Add a board	Implement backend build, programming, measurement, status, cleanup, and capability reporting.
Add a policy	Reference declared task and hardware metrics without changing workload or backend code.

Metric availability is target-dependent. Backends report the memory, latency, energy, cadence, deadline, and status fields they support, and mark unsupported fields unavailable. Adding a board still requires integrating that board’s toolchain, programming path, telemetry, and measurement support, but that work stays inside the backend module. Policies and logs continue to consume normalized metrics output.

3.3 Adding Workloads and Model Families

The ML-side boundary in CREST has three separable pieces: dataset, task, and model family. These pieces can be replaced together or independently. The dataset defines examples and metadata. The task defines what the model should predict and how quality is measured. The model family defines the model architecture search space and builds each candidate. After that, candidates from the new modules follow the same CREST workflow as built-in candidates.

Users do not always need to replace the whole workload stack. If the task and model family stay fixed, adding a dataset only requires a dataset adapter that provides examples, splits, input shape, data type, calibration data, and optional cadence metadata. If the dataset and task stay fixed, adding a model family only requires a model-family implementation that defines the search space, builds candidates, and exports a model compatible with the task output. A full new workload usually adds all three pieces: a dataset adapter, a task definition, and a model family.

The UrbanSound8K case study in §5.3 is the full-workload case. It exercises all three workload-facing interfaces in Table 3 by replacing OxiOD odometry with prepared-feature audio classification. It also changes the task metric and policy from Pareto exploration to a scalar application score. The backend deployment path, HIL measurement path, and replay path stay fixed, so the study tests a new deployment decision rather than a separate NAS pipeline.

3.4 Runtime Schedules

CREST treats runtime schedule as part of the study definition because sensing deployments usually run as scheduled windows rather than saturated inference loops [10–12, 50]. Runtime schedule changes what “efficient” means. *Continuous inference* repeats inference with no intentional release interval and measures per-inference cost under a saturated loop. *Cadenced inference* releases one inference every period T . After inference completes, the board spends the remaining window in a scheduled sleep interval until the next release.

A real sensing window may include sensor sampling, feature handling, control logic, communication, inference, and low-power waiting [12, 42]. Schedule can change model selection because latency measures active execution time, while deployment energy is

Table 3: Independently replaceable workload-facing interfaces for adding a dataset, task, or model family.

Interface	User provides	CREST expects
Dataset	Examples, splits, input shape, data type, calibration data	Data and metadata usable by the task and export path
Task	Loss, training setup, output interpretation, task metrics	Normalized quality metrics for each candidate
Model family	Search space, candidate builder, export model	Concrete model for training, export, and HIL evaluation

Table 4: One-second phase-energy probe. Values are mean board energy in mJ over 50 trials.

Target	Sleep [mJ]	Active int [mJ]	Active FP [mJ]
Nano 33 BLE	24.8	46.7	43.9
Portenta M4	940.2	993.4	976.7
Portenta M7	932.6	1045.5	1020.3
STM32 N657	823.1	892.5	905.1

integrated board power over the whole window. A general sensing window can be written as the sum of its phase costs:

$$E_{\text{window}} = P_{\text{inf}} T_{\text{inf}} + P_{\text{not_inf}} T_{\text{not_inf}} + P_{\text{sleep}} T_{\text{sleep}} + E_{\text{wake}} \quad (1)$$

In each product, P is the board power during a phase and T is the time spent in that phase. Thus, $P_{\text{inf}} T_{\text{inf}}$ is inference energy, $P_{\text{not_inf}} T_{\text{not_inf}}$ is non-inference work energy, and $P_{\text{sleep}} T_{\text{sleep}}$ is sleep energy. E_{wake} accounts for wake and sleep transitions. Equation 1 shows that deployment energy is phase-composed rather than inference-only. Sleep still requires state retention and board overhead, and duty-cycled embedded systems must account for clocking, guard timing, and wake behavior [42]. Active inference can vary with memory movement, operator kernels, quantization dispatch, heterogeneous execution paths, and clock settings [26, 27, 31]. CREST therefore measures integrated board energy over the scheduled window rather than treating slack time as zero cost.

To ground this phase distinction, we ran a fixed-duration phase-energy probe on the evaluated target configurations. Each trial measured one second of board-level energy while the DUT either entered its sleep path or executed a simple active arithmetic loop. Table 4 reports the mean over 50 trials and shows the property needed for the schedule argument. Sleep consumes measurable board energy, and replacing sleep with active work changes the energy of the same one-second interval.

The evaluated CREST cadenced studies instantiate this model as a controlled two-phase window with inference followed by scheduled sleep until the next release.

$$E_{\text{window}} \approx E_{\text{inference}} + E_{\text{sleep}} \quad (2)$$

This controlled scope isolates the schedule effect tested in §5.2 while keeping workload, model family, backend, and measurement path fixed. Broader deployments can include additional non-model phases in the same window. In these studies, CREST records integrated two-phase window energy in cadenced-specific metrics but does not decompose it into separate phase terms.

Figure 4 contrasts continuous inference, which mostly measures repeated active inference and loop overhead, with cadenced measurement, which asks whether the candidate meets the release period and how much board energy the whole window costs. Under a fixed period, candidates can swap order because active duration, sleep behavior, and deadline telemetry change the selected design.

Post-hoc adjustment cannot fully replace scheduled measurement because it does not execute the backend’s real wait or sleep path, measure timing in context, or report deadline misses. CREST therefore exposes schedule directly to the HIL evaluation path.

When board-power measurement is available, energy in both runtime regimes is normalized from measured board power.

$$E_{\text{norm}} = \frac{1000}{N} \int_{t_0}^{t_1} P_{\text{board}}(t) dt \quad (3)$$

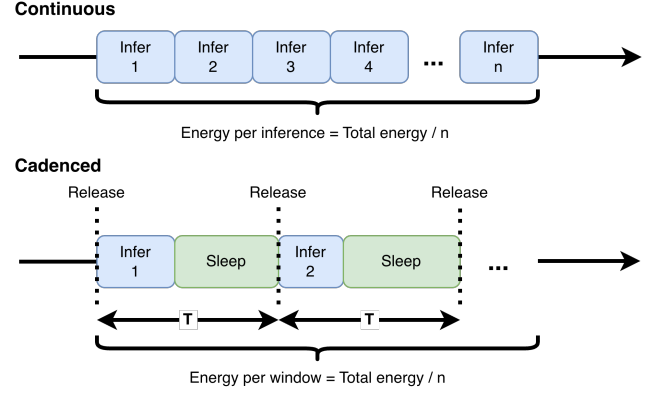


Figure 4: Runtime schedules in CREST. Continuous inference repeats inference without an intentional release interval. Cadenced inference releases one inference per period T and measures the scheduled window, including active inference and the following sleep interval.

Here, $P_{\text{board}}(t)$ is board power over the measured interval, N is completed invocations for continuous inference or scheduled releases for cadenced inference, and the factor of 1000 converts from joules to millijoules.

3.5 Connection to the Evaluation

The evaluation uses the system architecture above as an experimental control. CS1 (§5.1) varies policy and target backend while holding the odometry workload and model family fixed. CS2 (§5.2) varies runtime schedule on the same target, workload, and model family. CS3 (§5.3) changes the workload, model family, task metric, and scoring policy while reusing the backend interface, HIL path, and replay workflow.

These comparisons support the paper’s central claim. Deployment cost is a property of the architecture, target, schedule, workload, and policy together. Because CREST keeps those choices separate, the evaluation can attribute observed differences to deployment conditions rather than to a rewritten pipeline.

4 Implementation

CREST is implemented so that a study can change one part of the deployment setup without rewriting the NAS loop. The same client samples candidates, builds and evaluates them as configured, asks for hardware measurements when needed, and records the returned metrics. Board-specific build, programming, memory reporting, timing, and power-measurement code stays in the backend, addressing ❸. Runtime mode chooses whether the backend measures continuous or cadenced execution, addressing ❹. The same search, logging, replay, and metric format are reused when a study changes workload, board, runtime schedule, or policy, addressing ❺. This lets the evaluation change one factor at a time and compare the resulting model choices.

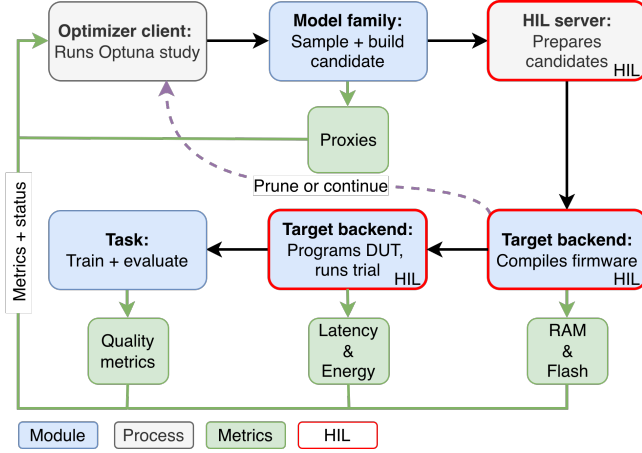


Figure 5: Per-trial implementation path in CREST. The optimizer client samples a candidate, task and model-family modules produce host-side quality and proxy metrics, and trials use HIL only when compile-time or runtime measurements are required. Target backends return memory, latency, energy, and status when available through the same feedback path used for pruning, feasibility checks, logging, and optimizer updates.

4.1 Search Engine and Trial Orchestration

CREST uses Optuna [2] as the optimizer. Single-objective studies use the TPE sampler [13] with multivariate sampling after startup trials. Multi-objective studies use NSGA-II [16] with a configured population size. CREST wraps each Optuna trial with the same candidate-building, measurement, feasibility, and logging code used by the rest of the system. This lets Optuna choose the next candidate while CREST controls how candidates are trained, deployed, measured, rejected, and recorded.

At startup, CREST resolves the selected study configuration and rejects policy rules that reference unavailable task or system metrics. During a trial, the policy’s metric dependencies determine whether CREST stops after host-side metrics, collects compile-time resource reports, or requests hardware measurements, as shown in Figure 5.

CREST records measured candidates, policy-rejected candidates, and failures instead of dropping them from the logs. User-defined pruning and feasibility rules decide when a candidate is rejected. A candidate can also fail before measurement if it cannot compile, flash, allocate memory, or complete the runtime protocol. Transport, device-discovery, and backend setup failures can still abort a study because the measurement path itself is unavailable. In scalar studies, pruned candidates remain visible as pruned Optuna trials. In multi-objective studies, where Optuna does not support pruning, rejected candidates can receive direction-aware penalty values while the rejection reason remains in the trial log.

4.2 HIL Server and Candidate Preparation

The HIL server sits between the optimizer client and the target backend. For each HIL trial, the client sends the candidate hyper-parameters, runtime metadata, quantization mode, and optional

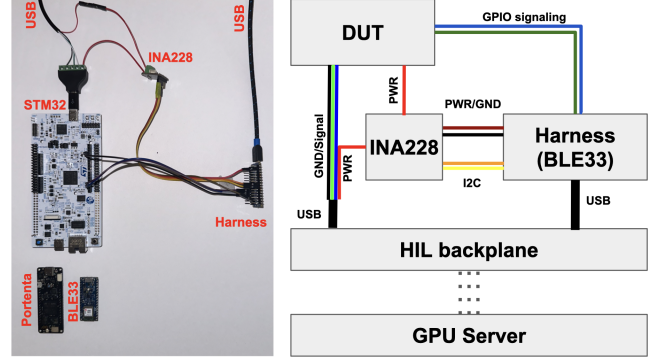


Figure 6: CREST HIL power-measurement setup. The DUT is powered through an INA228 monitor, and a separate harness MCU reads power telemetry over I2C while observing the DUT’s GPIO markers.

target options. The server resolves input metadata from the selected dataset and model context, builds the requested export model, checks the input shape, validates the task output, and sends the candidate to the selected backend.

The optimizer client and HIL server communicate through a ZeroMQ request-reply connection that carries candidate and runtime metadata to the server and returns metrics and status to the client. The two processes can run on the same host. They can also run on separate machines, with the optimizer on a GPU server or cloud VM and the HIL server on the machine connected to the DUT.

4.3 HIL Harness and Power Instrumentation

Figure 6 shows the HIL power-measurement setup. CREST adapts Bartoli et al.’s trigger-delimited TinyML energy-measurement method to a per-trial HIL NAS harness [12]. Their setup uses an oscilloscope to observe shunt voltage and DUT trigger signals. CREST instead powers the DUT through an off-the-shelf Adafruit INA228 breakout board [23]. In our setup, a separate Arduino Nano 33 BLE Sense harness MCU reads the INA228 over I2C using the Adafruit INA228 Arduino library [1] and observes DUT GPIO markers.

The GPIO markers define the measurement window. This lets CREST attach each energy value to a specific candidate, runtime schedule, and trial outcome without manual trace segmentation or signal processing to infer where inference occurred. Continuous-inference runs mark the repeated inference interval. Cadenced runs mark the scheduled window that includes inference and the following sleep interval.

The harness records board-level DUT supply measurements using a 15 mΩ shunt, 0.30 A expected-current range, 40.96 mV ADC range, 280 μs current conversion, 150 μs voltage conversion, and 64-sample averaging [1, 45]. At the GPIO-marked start of each measurement window, the harness resets the INA228 energy accumulator, and at the end marker it reads the accumulated energy value and normalizes it by the completed invocation count. These settings give CREST an accessible, low-cost power-measurement path. The Adafruit INA228 breakout costs about \$15, and the INA228 datasheet specifies sub-percent full-scale power measurement error [23, 45]. On coordinated DUT-plus-harness paths, harness energy is accepted only when DUT timing exists, harness completion

is observed, and DUT and harness run counts match. Harness-only paths rely on harness completion and harness telemetry.

4.4 Backend Implementations

We instantiate the target-backend interface on the Arduino Nano 33 BLE Sense (Cortex-M4) [5], the Arduino Portenta H7 (heterogeneous Cortex-M7 and Cortex-M4) [6], and the STM32 NUCLEO-N657X0-Q (Cortex-M55) [44]. Although these development boards use different vendor stacks, CREST routes their normalized metrics back to one shared NAS loop.

Table 5 shows how the evaluated backends populate hardware fields. Each backend owns the board-specific build, programming, measurement, status, and cleanup work.

The evaluated backends illustrate why target-specific code is isolated in backend modules. The Arduino-family targets share an `arduino-cli` build and upload path, but Portenta H7 requires target-core handling because its heterogeneous cores use different execution and measurement paths. STM32 N657 uses a separate ST Edge AI and ST toolchain path with different memory reports, programming tools, and telemetry. These differences affect how a candidate is built and measured, but they stay behind the target backend. Once the backend returns the fields it supports, the NAS loop, policy code, and replay code can consume the same field names across boards.

Runtime mode also resolves inside the backend. §3.4 defines continuous and cadenced inference. The implementation maps those modes to direct serial control, coordinated DUT-plus-harness telemetry, or harness-only telemetry. Cadenced telemetry is backend-dependent. The STM32 N657 backend reports more cadenced timing fields than the Arduino backends because the STM32 runtime exposes more schedule and sleep-state information. The evaluation therefore reports only the schedule metrics supported by the backend and harness path used in each study.

Each trial resolves one model architecture, quantization mode, and export variant before host TFLite evaluation or backend preparation. This keeps the host-evaluated and deployed candidates aligned because they share the same model architecture and quantization mode, while the export variant determines the concrete model artifact sent to the backend. Backends then perform device-specific conversion, staging, build, upload, and measurement.

4.5 Logging and Replay

CREST records the selected configuration, candidate payloads, per-trial CSV logs, and selected artifacts. Trial logs include task metrics, hyperparameters, objective metadata, feasibility metadata, quantization mode, hardware metrics, status fields, and cadenced telemetry

when present. Multi-objective studies export Pareto-front candidates for analysis and replay, while scalar studies track the best feasible candidates under the score. Replay runs reconstruct candidate payloads from study logs, reuse the same candidate-preparation and backend code as the HIL server, and record replay outputs.

5 Evaluation

We evaluate CREST on two sensing workloads across three representative MCUs (Arm Cortex-M). The first workload is inertial odometry on OxIOD [14], using a temporal convolutional network family similar to prior neural inertial odometry work [40]. The second workload is audio classification on UrbanSound8K [41], using a depthwise-separable CNN family in the lineage of compact MCU audio models [11, 35, 50]. The three boards are an Arduino Nano 33 BLE Sense (Cortex-M4), an Arduino Portenta H7 (heterogeneous Cortex-M7 and Cortex-M4), and an STM32 N657 (Cortex-M55), spanning roughly an order of magnitude in compute and the breadth of memory and accelerator configurations typical of MCU sensing. Where the Portenta’s heterogeneous cores are evaluated separately, we report them as separate CM7 and CM4 configurations. The same search-and-evaluation loop runs across all workload and target combinations through the interfaces, HIL boundary, and toolchain integrations described in §3 and §4.

Each NAS trial trains the candidate model for 55 epochs before deployment, and each HIL measurement reports latency and energy over 10 inference executions with the within-trial coefficient of variation typically below 1% for energy on all boards. Search uses Optuna [2], with budgets reported per study. For OxIOD, the task quality metric is aggregate odometry error,

$$\text{RMSE}_{\text{total}} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{v}_{x,i} - v_{x,i})^2} + \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{v}_{y,i} - v_{y,i})^2} \quad (4)$$

Continuous-inference studies report per-inference energy and latency, while cadenced studies additionally report normalized cadenced energy, active inference latency, cadenced phase timing, and deadline misses.

We organize the evaluation around three case studies. Case Study 1 (§5.1) tests the operation-count-based proxy assumption. Case Study 2 (§5.2) tests the continuous-inference evaluation assumption. Case Study 3 (§5.3) tests whether the same system can change workload, model family, task metric, target backend, and scoring policy to choose one deployable model for an application.

Table 5: Backend-specific extraction paths used to populate CREST’s normalized hardware metrics.

Backend	Toolchain	Memory extraction	Timing and energy extraction
Nano 33 BLE Sense	<code>arduino-cli</code>	Flash and RAM from Arduino CLI, arena from arena sweep	Latency from DUT serial and energy from harness when configured
Portenta H7	<code>arduino-cli</code> with CM7 helper for CM4	Flash and RAM from Arduino CLI, arena from arena sweep	CM7 latency from direct serial or harness, CM4 latency and energy from harness only
STM32 N657	ST Edge AI and ST toolchain	Arena from ST Edge AI, RAM from <code>arm-none-eabi-size</code> , flash from signed App binary, heap and stack from linker scripts, external weights from generated artifacts	DUT serial telemetry, harness energy, optional harness latency

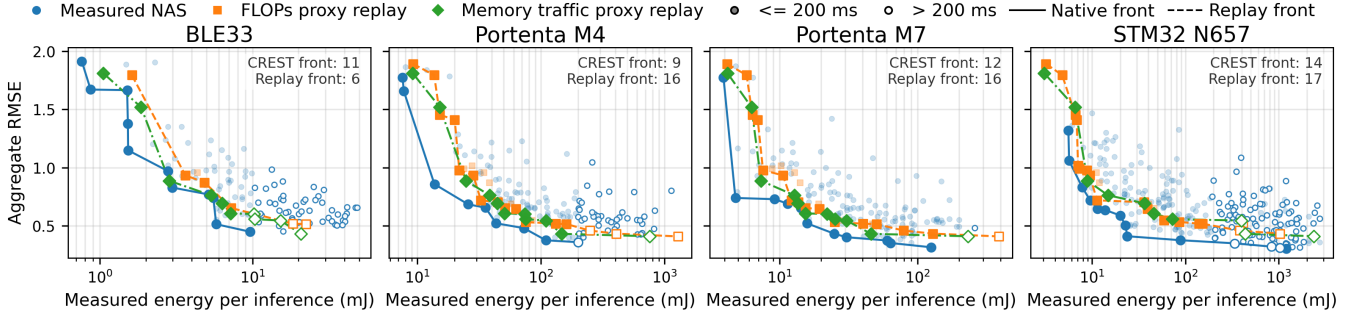


Figure 7: Static-proxy Pareto replay versus measured-energy NAS fronts. Each panel shows one target. The orange dashed front replays the FLOPs-proxy Pareto candidates on that target. The green dash-dot front replays the memory-traffic-proxy Pareto candidates. The solid blue front shows the target’s measured-energy NAS front. The x-axis is measured energy per inference on a log scale and the y-axis is aggregate RMSE, where lower is better. Filled markers meet the 200 ms latency threshold.

5.1 Case Study 1: CREST Outperforms Static-Proxy Ranking

CS1 evaluates whether static-proxy ranking selects the same hyperparameters as measured-energy HIL NAS, and whether the workload and model-family interfaces retarget across MCU backends through configuration choices alone.

Setup. We hold the workload fixed to OxIOD, the task to odometry regression, and the model family to TCN architectures. We vary the selection policy and target backend, comparing FLOPs-proxy and memory-traffic-proxy NAS runs against measured-energy NAS runs on BLE33, Portenta M4, Portenta M7, and STM32 N657 (Table 6). The measured runs optimize RMSE and measured energy. The FLOPs-proxy run optimizes `rmse_total` and `flops`. For each candidate m , let $O(m)$ be the operators in its frozen batch-size-one TensorFlow graph and $\phi(o)$ be TensorFlow’s profiler-reported float-operation count for operator o . The FLOPs proxy is

$$F(m) = \sum_{o \in O(m)} \phi(o) \quad (5)$$

$F(m)$ is computed from the frozen TensorFlow graph before any backend operation, with no per-target operator cost scaling.

The memory-traffic-proxy run optimizes `rmse_total` and the static byte count $T(m, q)$. For each Keras layer ℓ , let I_ℓ , O_ℓ , and W_ℓ be the scalar counts of its batch-size-one input activations, output activations, and unique weights, and let $b(q)$ be the scalar byte width under deployment quantization mode q . The static memory-traffic proxy is

$$T(m, q) = b(q) \sum_{\ell \in m} (I_\ell + O_\ell + W_\ell) \quad (6)$$

This proxy captures graph-level tensor traffic omitted by FLOPs, but it remains a static estimate that does not model cache behavior, kernel scheduling, or backend-specific memory placement. Memory-access and platform effects on MCU deployment cost have been observed in prior work [27, 30, 32]. For NAS, proxy error matters when it changes candidate ordering. A proxy can select a different Pareto front when it gives the wrong order to candidates with similar task error.

Each run uses Optuna’s NSGA-II sampler [2, 16] with a fixed trial budget. The TCN model family spans 8.1M hyperparameter combinations across filters, kernel sizes, dropout rates, skip-connection

choices, normalization choices, and dilation patterns. The static-proxy runs fix quantization to `int8_ptq`, while measured-energy runs additionally sample quantization mode and (for STM32) five CPU-clock presets [200, 300, 400, 600, 800] MHz. We expose CPU clock as a search axis only on STM32 because the Arduino deployment path used for the Nano 33 BLE Sense and Portenta H7 does not provide user-facing clock controls. We allocate 150 trials per run. STM32 uses 250 trials because its CPU-clock dimension multiplies the deployment space by 5×. Median trial durations are 6.5–8.3 minutes across runs. Training and candidate preparation dominate trial time, with HIL measurement contributing the smaller fraction.

Replay decouples architecture selection from deployment evaluation. We replay each static-proxy Pareto front on each measured target using the same backend measurement interface used by the measured NAS runs, so proxy-selected candidates and measured-energy candidates are evaluated through the same HIL path. Replay reuses each proxy-selected candidate’s representation and runs it through the target backend without invoking the search. A replayed point is dominated when a measured-energy NAS candidate on the same target has no higher energy and no higher `rmse_total`, and is strictly better in at least one objective. Candidates without

Table 6: CS1 retargets the same OxIOD TCN study across proxy and measured-HIL configurations. The workload, task metric, model-family interface, candidate representation, and trial-log schema are fixed. The objective and target configuration change.

Run	Backend path	Policy objective	Trials
FLOPs proxy	No HIL target selection	<code>rmse_total</code> with <code>flops</code> , fixed <code>int8_ptq</code>	150
Memory traffic proxy	No HIL target selection	<code>rmse_total</code> with <code>memory_traffic_bytes</code> , fixed <code>int8_ptq</code>	150
BLE33	Arduino BLE33	<code>rmse_total</code> with measured energy, float/ <code>int8_ptq</code>	150
Portenta M4	Arduino Portenta CM4	Same measured policy as BLE33	150
Portenta M7	Arduino Portenta CM7	Same measured policy as BLE33	150
STM32 N657	STM32 N657	Same measured policy plus five CPU clocks	250

valid HIL measurements are excluded from the dominance comparison. We also compute a nearest-front RMSE energy gap. Each replayed point is paired with the measured-energy Pareto point of closest `rmse_total`. The median absolute RMSE mismatch for this pairing is 0.013–0.044 across boards for FLOPs-proxy replay and 0.036–0.051 for memory-traffic-proxy replay.

Results. The FLOPs front contains 23 candidates, so replaying it on four targets gives 92 target measurements. Of these, 76 produced valid HIL measurements (Table 7). BLE33 accounts for the largest share of replay failures because its 256 KB SRAM and 1 MB flash budgets [5] reject 15 of 23 proxy-selected candidates that compile and run on the larger Portenta and STM32 boards. FLOPs is therefore not merely noisy. It can select infeasible deployments because it ignores fit-related activation and weight memory. The memory-traffic front contains 12 candidates and produces 47 valid measurements out of 48 across targets, avoiding most FLOPs feasibility failures but not recovering the measured-energy front.

Figure 7 compares valid replay fronts. Measured-energy NAS dominates 49 of 55 FLOPs-proxy replay Pareto-front points across the four targets, and only 47 of 55 valid replay-front points meet the 200 ms latency threshold. Using nearest-front RMSE matching, the measured-energy front reduces median energy by 35–47% per board for FLOPs-proxy replay, with a cross-board median of 41.7% (Table 7). The median absolute RMSE mismatch in the pairing is small relative to the 0.3–2.0 RMSE range of front candidates, so the energy gap reflects genuine architectural differences rather than RMSE-window noise.

Memory-traffic proxy selection narrows the failure mode but does not eliminate it. Measured-energy NAS dominates 37 of 42 memory-traffic-proxy replay Pareto-front points, and 33 of 42 valid replay-front points meet the 200 ms threshold. At nearest-front RMSE, measured-energy NAS reduces median energy by 23.7%, 44.6%, 37.1%, and 64.4% on BLE33, Portenta M4, Portenta M7, and STM32 N657, with a cross-board median of 40.8%. The remaining gap reflects target-specific behavior that static graph-level proxies cannot capture. This target-specific behavior includes platform effects outside the model graph, such as memory placement, caches, board power behavior, and runtime power management.

CREST Impact. CS1 demonstrates a controlled cross-target comparison that earlier hardware-aware NAS systems cannot perform within one framework. CREST runs the same workload, task, and model family under static-proxy ranking and measured-energy

Table 7: CS1 per-board replay results. Valid counts are out of the source proxy front: 23 candidates for FLOPs and 12 for memory traffic. Dominated and ≤ 200 ms counts are out of valid replay-front points per board.

Proxy	Board	Valid	Dominated	≤ 200 ms	Energy gap
FLOPs	BLE33	8/23	6/6	4/6	45.8%
FLOPs	Portenta M4	23/23	16/16	13/16	46.7%
FLOPs	Portenta M7	23/23	15/16	15/16	37.6%
FLOPs	STM32 N657	22/23	12/17	15/17	35.3%
Mem. traffic	BLE33	11/12	7/10	6/10	23.7%
Mem. traffic	Portenta M4	12/12	11/11	10/11	44.6%
Mem. traffic	Portenta M7	12/12	10/11	10/11	37.1%
Mem. traffic	STM32 N657	12/12	9/10	7/10	64.4%

HIL search, then replays each across four Cortex-M targets without changing the workload, model-family, or measurement code. The comparison exposes deployment failures and energy gaps that proxy-only NAS systems miss because their selection mechanisms do not observe per-target memory, scheduling, or operator-cost differences. CREST turns scoring policy and target backend into configurable axes a user can audit rather than implementation commitments baked into the pipeline.

5.2 Case Study 2: CREST Reveals What Continuous-Inference Evaluation Misses Under Cadenced Deployment

CS2 evaluates whether continuous-inference measurement is sufficient for a sensing deployment that releases one inference per fixed window. In a cadenced deployment, the application sets the period T . A model is timing-feasible if its active latency L_{model} fits within that period. The remaining time is scheduled sleep, so a simplified two-phase window has energy

$$E_{\text{window}} = P_{\text{active}}L_{\text{model}} + P_{\text{sleep}}(T - L_{\text{model}}), \quad L_{\text{model}} \leq T. \quad (7)$$

This makes latency a deadline constraint and energy a full-window quantity. The phase probe in Table 4 gives the scale of the active/sleep tradeoff. On STM32 N657, one second of sleep costs 823.1 mJ, while one second of active FP work costs 905.1 mJ. For the 200 ms period used in CS2, this gives a sleep-only baseline of 164.6 mJ, and each additional 10 ms spent active instead of asleep adds about 0.82 mJ in the simplified probe. The relevant cost then becomes per-window energy instead of per-inference active energy, so we test whether candidates selected under one runtime remain Pareto-competitive when replayed under the other.

Setup. We focus on STM32 N657 because its backend reports active latency, scheduled-window timing, per-window energy, deadline misses, and sleep telemetry. The two NAS runs share the STM32 backend, OxIOD task, TCN model family, input mode, candidate representation, search space, HIL path, and the same five CPU-clock presets [200, 300, 400, 600, 800] MHz. They differ only in runtime schedule and objective: the continuous run optimizes `rmse_total` and per-inference energy, while the cadenced run uses a representative 200 ms sensing period and optimizes `rmse_total` and per-window energy. In cadenced mode, post-inference slack is used for sleep when the interval is long enough; short slack intervals are handled by waiting awake until the next release. Cross-runtime replay decouples the selected architecture set from the runtime used to evaluate it by measuring each frontier in the other runtime without retraining.

For cadenced reporting, we apply a deployment-valid audit rule requiring zero deadline misses, valid window timing, positive window energy, and active latency within the 200 ms budget. The cadenced run produced 369 non-pruned measurements, of which 260 satisfied the search-time feasibility label before audit. The audit removes 14 deadline-miss rows, leaving 246 strict-valid cadenced candidates for the cadenced panel and replay comparisons. Replay points are included only when the target runtime measurement completed successfully and satisfied the panel’s feasibility rule. Pareto fronts in each panel are recomputed in that panel’s objective space after filtering, so the dashed fronts show transfer under the

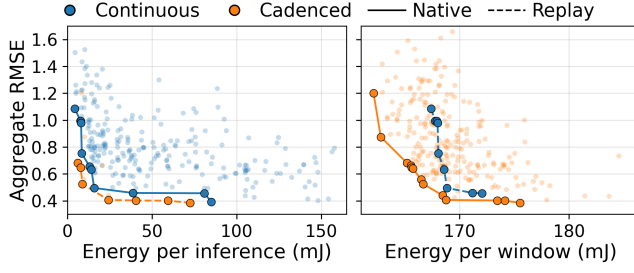


Figure 8: CS2 cross-runtime Pareto comparison for STM32/OxIOD. The left panel evaluates both selected sets under continuous-inference energy per inference. The right evaluates both under cadenced energy per window. Solid lines are native NAS fronts, and dashed lines are replays.

target runtime rather than source-runtime rank. CS2 energy gaps use the same nearest-front RMSE matching as CS1.

Results. Figure 8 shows that runtime schedule changes the measured Pareto geometry for STM32/OxIOD. When replayed under the cadenced schedule, the continuous-inference front yields 10 strict-valid candidates and 9 replay-front points, all of which are dominated by the native cadenced front. No cadenced-front points are dominated by the continuous-inference replay front, and the combined cadenced-view frontier contains only cadenced-selected candidates. At nearest-front RMSE, the native cadenced front saves a median 3.6 mJ per window, or 2.1%, relative to the continuous-inference replay front.

The reverse transfer is also nontrivial. The native continuous-inference study contributes 252 valid candidates and 11 front points, while replaying the cadenced-selected front contributes 13 measured candidates and 7 front points. The cadenced replay front dominates 9 of the 11 native continuous-inference front points and uses a median 12.4 mJ less active energy at nearest-front RMSE, a 44.1% reduction. Changing the schedule therefore steers search toward a different hyperparameter region rather than only adding scheduled sleep energy.

CREST Impact. CS2 demonstrates schedule as a first-class search-time configuration and benchmark axis. Runtime schedule is a model-selection dimension, not only a reporting dimension. CREST keeps the workload, model family, backend, candidate representation, replay path, and metric schema fixed while changing the runtime schedule. In this STM32/OxIOD study, a front selected under benchmark-style continuous inference does not transfer to the deployment schedule. Without schedule-aware HIL search and cross-runtime replay, this frontier shift would be hidden behind a single active-energy measurement.

5.3 Case Study 3: Application-Level Scoring Selects Target-Specific Models

CS1 and CS2 use Pareto fronts to show how proxy objectives and runtime schedule change model selection. CS3 tests whether the same CREST workflow can support application-level deployment selection after changing the workload, model family, task metric, target backend, and scoring policy. We focus on prepared-feature audio classification as a driving example, using UrbanSound8K with

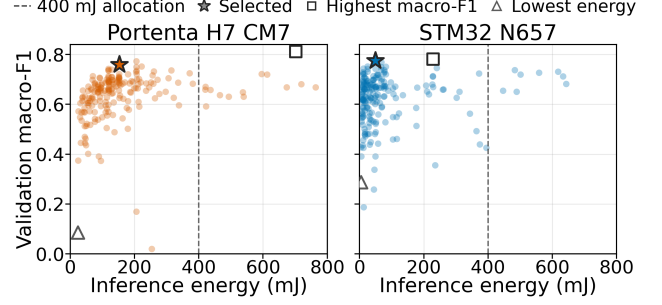


Figure 9: CS3 score-based prepared-feature UrbanSound8K DS-CNN selection on two backends. Each point is a feasible scored trial, plotted by validation macro-F1 and HIL-measured active inference energy. The dashed line marks the 400 mJ classifier allocation used in the score.

a DS-CNN model family. CS3 asks which model hyperparameters should be deployed for a prepared-feature classifier, an application energy budget, and a target backend. We replace Pareto exploration with a scalar score over validation macro-F1 and measured inference energy. We use UrbanSound8K as a prepared-feature classification workload. This keeps the comparison focused on model inference rather than microphone capture or feature extraction.

Setup. We replace odometry regression with prepared-feature UrbanSound8K audio classification. The new modules add cached log-mel inputs, a sound-classification task, and DS-CNN model-family adapters. This changes the workload, model family, task metric, and policy. It reuses the orchestration loop, HIL server, backend metric schema, feasibility handling, scoring machinery, and replay path. UrbanSound8K provides 10 predefined folds and recommends using them for cross-validation rather than reshuffling examples [41]. CS3 uses a fixed search split, with folds 1–8 for training and fold 9 for validation, so validation macro-F1 can rank candidate configurations during HIL search. The fixed split gives HIL search a stable validation signal. Fold 10 remains outside the search and can be used for later benchmark-style or deployment-specific checks after search.

We run the same single-objective audio search on two backends, Portenta H7 CM7 and STM32 N657, each configured for 200 feasible scored candidates with up to 300 total trial attempts. For hardware measurement, candidates run on prepared-feature tensors matching the cached log-mel shape, so reported latency and energy isolate model inference rather than microphone capture or feature extraction costs.

For the scalar policy, we use a portable field-node budget in which one prepared log-mel feature window is classified every 2 s during a 72 h session. A 20,000 mAh USB power bank gives roughly a 0.8 W usable system envelope, or 1600 mJ per 2 s window [3, 4, 46]. We allocate 25% of this window to classifier inference, giving a 400 mJ policy budget while reserving the rest for sensing, feature extraction, storage, control, and other overheads. This budget is a user policy setting, not a universal deployment constant. We use it as an example to show how CREST combines validation quality with measured embedded inference cost.

The score ranks validation quality against measured embedded model cost.

$$\text{score} = \text{MacroF1}_{\text{val}} - 0.10 \cdot \frac{E_{\text{inf}}}{E_{\text{inf_budget}}} \quad (8)$$

The 0.10 coefficient is a user-defined quality-energy exchange rate. A candidate at the full 400 mJ budget pays a 0.10 validation macro-F1 penalty, while a candidate at half the budget pays 0.05. Latency and memory are feasibility constraints, not score terms. Because the coefficient and budget are policy choices, we test whether the selected deployment depends on this setting by re-scoring the completed audio trials over inference budgets from 100–1200 mJ and positive energy penalties from 0.025–0.30.

Results. Figure 9 shows the feasible scored trials for each board. The score does not choose the maximum-validation-macro-F1 candidate on either board or the lowest-energy candidate. On Portenta H7 CM7, the score-selected model reaches 0.760 validation macro-F1 at 152.5 mJ per inference. The maximum-validation-macro-F1 candidate reaches 0.812 validation macro-F1 at 701.9 mJ. On STM32 N657, the score-selected model nearly ties the maximum-validation-macro-F1 candidate (0.776 vs. 0.781) while reducing measured inference energy from 228.5 mJ to 49.3 mJ. Both boards reduce energy by approximately 78%, but the validation macro-F1 cost differs by nearly an order of magnitude (0.052 on Portenta vs. 0.006 on STM32). The score makes this quality-energy tradeoff explicit.

The score-based conclusion is not unique to the 400 mJ, 0.10 policy setting. In the sensitivity analysis, the selected hyperparameters change on Portenta, while STM32 selects the same architecture for all positive energy penalties and for all budgets except the strictest 100 mJ setting. Across both boards, all budget settings and 11 of 12 positive-penalty settings select lower-energy candidates than the maximum-validation-macro-F1 candidate. Thus, the exact policy changes which model is selected. The main conclusion is stable. Score-aware selection usually trades small validation-quality losses for large measured-energy savings.

The two boards also select different DS-CNN shapes from the same search space. The Portenta run selects a compact classifier that downsamples aggressively. The STM32 run selects a wider classifier that keeps more capacity later in the network. When each score-selected model architecture is replayed on the other board, both become more expensive (Figure 10). The STM32-selected model architecture rises from 49.3 to 204.5 mJ on Portenta, a 4.1 $\times$ increase, with latency increasing from 45.7 to 179.5 ms. The Portenta-selected architecture rises from 152.5 to 218.7 mJ on STM32, a 1.4 $\times$ increase, with latency increasing from 133.0 to 230.8 ms. Both replays remain within the 2 s decision deadline. The STM32-selected wider classifier is consistent with a board that can use the STM32N657's Cortex-M55 MVE/Helium vector support for ML/DSP kernels [8, 44]. It pays a larger replay penalty when moved to Portenta than the compact Portenta-selected classifier pays when moved to STM32.

CREST Impact. CS3 shows that CREST supports controlled deployment selection, not only Pareto-front analysis. A user provides a workload, target backend, and scoring policy. CREST then searches using a fixed validation signal and measured board energy. The selected model architectures differ from the maximum-validation-macro-F1 and lowest-energy candidates. They differ by board and become more expensive under cross-board replay. Thus, the best

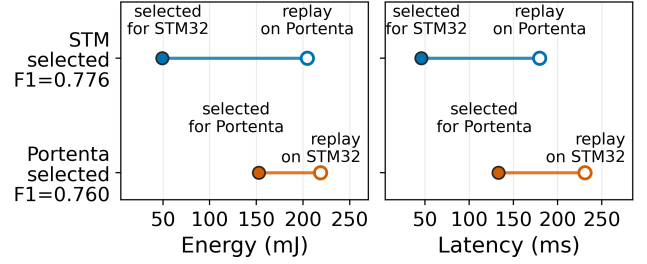


Figure 10: CS3 cross-board replay of score-selected audio model architectures. Each row is one selected DS-CNN, with source validation macro-F1 fixed in the row label. Filled endpoints are native measurements, open endpoints are replays on the opposite backend.

deployment choice depends jointly on the workload, policy, and target backend chosen for deployment.

6 Discussion and Conclusion

CS1, CS2, and CS3 show that deployment cost is a property of the model architecture, target, schedule, workload, and policy together. Static-proxy ranking misorders measured-energy fronts, continuous-inference selection does not transfer cleanly to censored deployment, and the same application-level score selects different audio models on different boards.

The HIL NAS runs each take 17–35 hours per board. This cost makes controlled comparison important. If each study used a different orchestration loop, HIL boundary, measurement path, candidate representation, or log format, differences between results could come from the pipeline rather than the deployment variable being tested. CREST keeps these pieces fixed across runs that differ only in the intended configuration change, allowing each case study to attribute observed differences to deployment conditions rather than a rewritten pipeline.

The case studies use controlled measurement slices to isolate the deployment variable under test: CS1 varies selection policy and backend, CS2 varies runtime schedule, and CS3 varies workload and application policy. This structure lets replay attribute frontier shifts to the configured deployment axis rather than to changes in orchestration, logging, or measurement code. The same boundary also defines future extensions, including phase-tagged end-to-end workloads that cover sensor, feature, inference, and communication phases, predictor-assisted searches trained from CREST HIL logs to reduce measurement cost, and replicated final-candidate evaluations when HIL budgets allow.

We presented CREST, an open-source HIL NAS framework for MCU sensing models. Across inertial odometry and audio classification, CREST shows that deployment cost is not captured by a static proxy, a single board, or a continuous-inference measurement. By keeping the optimizer, HIL measurement boundary, logging, and replay workflow fixed while varying workload, target backend, runtime schedule, and scoring policy, CREST makes these deployment effects experimentally separable. The right architecture is a property of the workload, target backend, runtime schedule, and policy together.

References

- [1] [n. d.]. *adafruit/Adafruit_INA228: Arduino library for INA228 I2C Power Monitor*. https://github.com/adafruit/Adafruit_INA228
- [2] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A Next-generation Hyperparameter Optimization Framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, 2623–2631. doi:10.1145/3292500.3330701
- [3] Anker. [n. d.]. *Anker Prime Power Bank (20K, 200W)*. <https://www.anker.com/products/a1336-20000mah-power-bank>
- [4] Anker Support. [n. d.]. *Why Is the Rated Capacity Marked on a Power Bank Lower Than Expected?* <https://service.anker.com/article-description/Why-Is-the-Rated-Capacity-Marked-on-a-Power-Bank-Lower-Than-Expected>
- [5] Arduino [n. d.]. *Arduino Nano 33 BLE Sense*. Arduino. <https://docs.arduino.cc/resources/datasheets/ABX00031-datasheet.pdf>
- [6] Arduino 2026. *Arduino Portenta H7 Collective Datasheet*. Arduino. <https://docs.arduino.cc/resources/datasheets/ABX00042-ABX00045-ABX00046-datasheet.pdf>
- [7] Arm. 2026. *Arm Cortex-M55*. <https://www.arm.com/products/silicon-ip-cpu/cortex-m/cortex-m55>
- [8] Arm. 2026. *Arm Helium Technology*. <https://www.arm.com/technologies/helium>
- [9] Colby Banbury, Vijay Janapa Reddi, Peter Torelli, Jeremy Holleman, Nat Jeffries, Csaba Kiraly, Pietro Montino, David Kanter, Sebastian Ahmed, Danilo Pau, Urmish Thakker, Antonio Torrini, Peter Warden, Jay Cordaro, Giuseppe Di Guglielmo, Javier Duarte, Stephen Gibellini, Videet Parekh, Honson Tran, Nhan Tran, Niu Wenxu, and Xu Xuesong. 2021. MLPerf Tiny Benchmark. NeurIPS. doi:10.48550/ARXIV.2106.07597 Version Number: 4.
- [10] Colby Banbury, Chuteng Zhou, Igor Fedorov, Ramon Matas, Urmish Thakker, Dibakar Gope, Vijay Janapa Reddi, Matthew Mattina, and Paul Wharmough. 2021. MicroNets: Neural Network Architectures for Deploying TinyML Applications on Commodity Microcontrollers. In *Proceedings of Machine Learning and Systems*, A. Smola, A. Dimakis, and I. Stoica (Eds.), Vol. 3. 517–532. https://proceedings.mlsys.org/paper_files/paper/2021/file/c4d41d9619462c534b7b61d1f72385e-Paper.pdf
- [11] Pietro Bartoli, Tommaso Bondini, Christian Veronesi, Andrea Giudici, Niccolò Antonello, and Franco Zappa. 2025. End-to-End Efficiency in Keyword Spotting: A System-Level Approach for Embedded Microcontrollers. In *2025 IEEE SENSORS*. 1–4. doi:10.1109/SENSORS59705.2025.11330257 ISSN: 2168-9229.
- [12] Pietro Bartoli, Christian Veronesi, Andrea Giudici, David Siorpaes, Diana Trojaniello, and Franco Zappa. 2025. Benchmarking Energy and Latency in TinyML: A Novel Method for Resource-Constrained AI. In *2025 International Joint Conference on Neural Networks (IJCNN)*. 1–8. doi:10.1109/IJCNN64981.2025.11228997
- [13] James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. 2011. Algorithms for Hyper-Parameter Optimization. In *Advances in Neural Information Processing Systems*, J. Shawe-Taylor, R. Zemel, P. Bartlett, F. Pereira, and K. Weinberger (Eds.), Vol. 24. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2011/file/86e8f7ab32cfd12577bc2619bc635690-Paper.pdf
- [14] Changhao Chen, Peijun Zhao, Chris Xiaoxuan Lu, Wei Wang, Andrew Markham, and Niki Trigoni. 2018. OxiOD: The Dataset for Deep Inertial Odometry. doi:10.48550/ARXIV.1809.07491 Version Number: 1.
- [15] Robert David, Jared Duke, Advait Jain, Vijay Janapa Reddi, Nat Jeffries, Jian Li, Nick Kreger, Ian Nappier, Meghna Natraj, Shlomi Regev, Rocky Rhodes, Tiezheng Wang, and Pete Warden. 2021. TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems. In *Proceedings of the 4th MLSys Conference*. San Jose, CA, USA.
- [16] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan. 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation* 6, 2 (2002), 182–197. doi:10.1109/4235.996017
- [17] Andrea Mattia Garavagno, Edoardo Ragusa, Antonio Frisoli, and Paolo Gastaldo. 2024. An Affordable Hardware-Aware Neural Architecture Search for Deploying Convolutional Neural Networks on Ultra-Low-Power Computing Platforms. *IEEE Sensors Letters* 8, 5 (2024), 1–4. doi:10.1109/LSNS.2024.3387056
- [18] René Groh and Andreas M. Kist. 2023. End-to-end Evolutionary Neural Architecture Search for Microcontroller Units. In *2023 IEEE International Conference on Omni-layer Intelligent Systems (COINS)*. IEEE, Berlin, Germany, 1–7. doi:10.1109/COINS57856.2023.10189194
- [19] Mohammad Amin Hasanpour, Mikkel Kirkegaard, and Xenofon Fafoutis. 2025. EdgeMark: An automation and benchmarking system for embedded artificial intelligence tools. *Journal of Systems Architecture* 167 (Oct. 2025), 103488. doi:10.1016/j.sysarc.2025.103488
- [20] Lennart Heim, Andreas Biri, Zhongnan Qu, and Lothar Thiele. 2021. Measuring what Really Matters: Optimizing Neural Networks for TinyML. doi:10.48550/arXiv.2104.10645 arXiv:2104.10645 [cs.LG].
- [21] Syed Monowar Hossain, Timothy Hnat, Nazir Saleheen, Nusrat Jahan Nasrin, Joseph Noor, Bo-Jhang Ho, Tyson Condie, Mani Srivastava, and Santosh Kumar. 2017. mCerebrum: A Mobile Sensing Software Platform for Development and Validation of Digital Biomarkers and Interventions. In *Proceedings of the 15th ACM Conference on Embedded Network Sensor Systems*. ACM, Delft Netherlands, 1–14. doi:10.1145/3131672.3131694
- [22] Shawn Hymel, Colby Banbury, Daniel Situnayake, Alex Elum, Carl Ward, Mat Kelcey, Mathijs Baaijens, Mateusz Majchrzycki, Jenny Plunkett, David Tischler, Alessandro Grande, Louis Moreau, Dmitry Maslov, Artie Beavis, Jan Jongboom, and Vijay Janapa Reddi. 2023. Edge Impulse: An MLOps Platform for Tiny Machine Learning. doi:10.48550/arXiv.2212.03332 arXiv:2212.03332 [cs].
- [23] Adafruit Industries. [n. d.]. *Adafruit INA228 - I2C 85V, 20-bit High or Low Side Power Monitor*. <https://www.adafruit.com/product/5832>
- [24] Tomoyoshi Kimura, Jinyang Li, Tianshi Wang, Yizhuo Chen, Ruijie Wang, Denizhan Kara, Maggie Wigness, Joydeep Bhattacharyya, Mudhakar Srivatsa, Shengzhong Liu, Mani Srivastava, Suhag Diggavi, and Tarek Abdelzaher. 2024. VibroFM: Towards Micro Foundation Models for Robust Multimodal IoT Sensing. In *2024 IEEE 21st International Conference on Mobile Ad-Hoc and Smart Systems (MASS)*. 10–18. doi:10.1109/MASS62177.2024.00014
- [25] Tobias King, Yexu Zhou, Tobias Röddiger, and Michael Beigl. 2025. MicroNAS for memory and latency constrained hardware aware neural architecture search in time series classification on microcontrollers. *Scientific Reports* 15, 1 (March 2025), 7575. doi:10.1038/s41598-025-90764-z
- [26] Liangzhen Lai, Naveen Suda, and Vikas Chandra. 2018. CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs. arXiv:1801.06601 [cs.NE] doi:10.48550/arXiv.1801.06601
- [27] Liangzhen Lai, Naveen Suda, and Vikas Chandra. 2018. Not All Ops Are Created Equal!. In *SysML Conference*. doi:10.48550/arXiv.1801.04326 eprint: 1801.04326.
- [28] Chaojian Li, Zhongzhi Yu, Yonggan Zhang, Yang Zhao, Haoran You, Qixuan Yu, Yue Wang, and Yingyan Celine Lin. 2021. HW-NAS-Bench: Hardware-Aware Neural Architecture Search Benchmark. doi:10.48550/ARXIV.2103.10584 Version Number: 2.
- [29] Edgar Liberis, Łukasz Dudziak, and Nicholas D. Lane. 2020. μ NAS: Constrained Neural Architecture Search for Microcontrollers. doi:10.48550/arXiv.2010.14246 arXiv:2010.14246 [cs].
- [30] Ji Lin, Wei-Ming Chen, Yujun Lin, John Cohn, Chuang Gan, and Song Han. 2020. MCUNet: tiny deep learning on IoT devices. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA.
- [31] Hao Liu, Qing Wang, and Marco Zuniga. 2026. InstMeter: An Instruction-Level Method to Predict Energy and Latency of DL Model Inference on MCUs. doi:10.48550/ARXIV.2603.04134 Version Number: 1.
- [32] Ningning Ma, Xiangyu Zhang, Hai-Tao Zheng, and Jian Sun. 2018. ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design. doi:10.48550/ARXIV.1807.11164 Version Number: 1.
- [33] Ashitabh Misra, Nurani Saoda, and Tarek Abdelzaher. 2025. Latency-Constrained Input-Aware Quantization of Time Series Inference Workflows at the Edge. In *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*. 1–10. doi:10.1109/INFOCOM55648.2025.11044709 ISSN: 2641-9874.
- [34] Filippo Naccari, Ivana Guarneri, Salvatore Curti, and Alberto Amilcare Savi. 2020. Embedded Acoustic Scene Classification for Low Power Microcontroller Devices. In *Proceedings of the Detection and Classification of Acoustic Scenes and Events 2020 Workshop (DCASE2020)*. Tokyo, Japan, 105–109. https://dcase.community/documents/workshop2020/proceedings/DCASE2020Workshop_Naccari_44.pdf
- [35] Jon Nordby. 2019. *Environmental Sound Classification on Microcontrollers using Convolutional Neural Networks*. Master's thesis. Norwegian University of Life Sciences.
- [36] Anas Osman, Usman Abid, Luca Gemma, Matteo Perotto, and Davide Brunelli. 2021. TinyML Platforms Benchmarking. doi:10.48550/ARXIV.2112.01319 Version Number: 1.
- [37] Ye Qiao, Haocheng Xu, Yifan Zhang, and Sitao Huang. 2025. MONAS: Efficient Zero-Shot Neural Architecture Search for MCUs. In *2025 International Joint Conference on Neural Networks (IJCNN)*. IEEE. doi:10.1109/IJCNN64981.2025.11227909
- [38] Bidyut Saha, Riya Samanta, Soumya K. Ghosh, and Ram Babu Roy. 2024. TinyT-NAS: GPU-Free, Time-Bound, Hardware-Aware Neural Architecture Search for TinyML Time Series Classification. doi:10.48550/ARXIV.2408.16535 Version Number: 1.
- [39] Swapnil Sayan Saha, Sandeep Singh Sandha, Mohit Aggarwal, Brian Wang, Liying Han, Julian De Gortari Briseno, and Mani Srivastava. 2024. TinyNS: Platform-aware Neurosymbolic Auto Tiny Machine Learning. *ACM Transactions on Embedded Computing Systems* 23, 3 (May 2024), 1–48. doi:10.1145/3603171
- [40] Swapnil Sayan Saha, Sandeep Singh Sandha, Luis Antonio Garcia, and Mani Srivastava. 2022. TinyOdom: Hardware-Aware Efficient Neural Inertial Navigation. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 6, 2 (July 2022), 1–32. doi:10.1145/3534594
- [41] Justin Salamon, Christopher Jacoby, and Juan Pablo Bello. 2014. A Dataset and Taxonomy for Urban Sound Research. In *Proceedings of the 22nd ACM international conference on Multimedia*. ACM, Orlando Florida USA, 1041–1044. doi:10.1145/2647868.2655045
- [42] Thomas Schmid, Roy Shea, Zainul Charbiwala, Jonathan Friedman, Mani B. Srivastava, and Young H. Cho. 2010. On the interaction of clocks, power, and synchronization in duty-cycled embedded sensor nodes. *ACM Trans. Sen. Netw.*

- 7, 3 (Oct. 2010). doi:10.1145/1807048.1807053
- [43] A. Sinha and A.P. Chandrakasan. 2000. Energy aware software. In *VLSI Design 2000. Wireless and Digital Imaging in the Millennium. Proceedings of 13th International Conference on VLSI Design*. IEEE Comput. Soc, Calcutta, India, 50–55. doi:10.1109/ICVD.2000.812584
- [44] STMicroelectronics. 2025. *STM32N6x5xx STM32N6x7xx Datasheet: Arm Cortex-M55-based MCU*. STMicroelectronics. <https://www.st.com/resource/en/datasheet/stm32n657z0.pdf>
- [45] Texas Instruments. [n. d.]. *INA228 85-V, 20-Bit, Ultra-Precise Power/Energy/Charge Monitor With I2C Interface*. <https://www.ti.com/lit/ds/symlink/ina228.pdf>
- [46] U.S. Department of Transportation. [n. d.]. *49 CFR §171.8: Watt-hour (Wh)*. https://www.law.cornell.edu/definitions/index.php?def_id=3ef5256c7320bac91820a83c19841917
- [47] Philipp Van Kempen, Rafael Stahl, Daniel Mueller-Gritschneider, and Ulf Schlichtmann. 2023. MLonMCU: TinyML Benchmarking with Fast Retargeting. In *Proceedings of the 2023 Workshop on Compilers, Deployment, and Tooling for Edge AI*. ACM, Hamburg Germany, 32–36. doi:10.1145/3615338.3618128
- [48] Yuwen Sun, Lucas Wanner, and Mani Srivastava. 2012. Low-cost estimation of sub-system power. In *2012 International Green Computing Conference (IGCC)*. IEEE, San Jose, CA, USA, 1–10. doi:10.1109/IGCC.2012.6322270
- [49] Li Lyna Zhang, Shihao Han, Jianyu Wei, Ningxin Zheng, Ting Cao, Yuqing Yang, and Yunxin Liu. 2021. nn-Meter: towards accurate latency prediction of deep-learning model inference on diverse edge devices. In *Proceedings of the 19th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys '21)*. Association for Computing Machinery, New York, NY, USA, 81–93. doi:10.1145/3458864.3467882
- [50] Yundong Zhang, Naveen Suda, Liangzhen Lai, and Vikas Chandra. 2018. Hello Edge: Keyword Spotting on Microcontrollers. doi:10.48550/arXiv.1711.07128 arXiv:1711.07128 [cs].